

Owner-Approved Provisioning of a Model-Key Broker

Binding the broker's configuration, receiving key and policy epoch before the model key leaves the owner

Imran Siddique · OPAQUE Systems · October 1, 2026

Technical report, version 1 · Software experiments

Abstract

A model owner can require workload attestation before a key broker releases a decryption key, and still lose control if the deployment operator can change the broker itself: its trust roots, its list of accepted model manifests, or the owner key it trusts. This report describes provisioning in the other direction. The owner treats the broker as a recipient. Before sending the model key, the owner appraises a signed attestation report that commits to the broker's launch measurement, a digest of the configuration the broker actually constructed, a fresh receiving key, the model identity and the policy epoch. A separate deployment approval pins the build artifacts and launch parameters. The receiver installs one owner-authenticated envelope, serves workloads, and refuses every path after retirement. We evaluate the implementation in the Weight Custody Manifest library, released in weight-custody-manifest 0.28.4 and later. All 84 existing tests pass, and removing the configuration or measurement check makes 5 and 3 specific tests fail. A retired receiver in its own process refused all four API paths, and an old envelope would not install in a replacement. Across separate owner processes sharing one epoch store, a stale live owner, a restarted stale owner and a same-epoch substitution were all refused, and a killed policy admission left the floor unchanged. Every number was reproduced on October 1, 2026 at the evaluated revision and at current main.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. All results are software results on one Windows host. Attestation reports are synthetic SNP-format reports signed by a generated certificate chain that has no relationship to a real AMD device. No SEV-SNP hardware, cloud resource, real model key or production broker was used. The report does not show that an approved image ran unchanged on confidential hardware, does not predict a launch measurement, and does not show that retired key material was erased. The rerun record, with commands, package versions and outputs, is in `experiments/results/rerun-2026-10-01/`.

1. Who checks the broker

Attestation-gated key release usually has one checker: the broker verifies the workload's quote and releases the key if the measurement matches. That works when the broker is trusted. In many deployments the broker runs on infrastructure the model owner does not control. The operator can restart it with a different certificate root, add a manifest identity to its allowlist, or swap the owner public key it uses to authenticate policy. The broker will then verify workload quotes correctly, against the wrong rules.

Approving a program name does not close that gap. The owner needs to know the exact inputs the running broker was constructed with, and needs the key to reach only that broker instance. The mechanism here makes the broker a provisioning recipient:

Owner question	What answers it
Is this the approved broker image?	The signed launch measurement must equal the owner's expected value.
Is it using the approved configuration?	The receiver hashes its actual constructor inputs and refuses to start if the digest differs from the owner policy; the digest is also bound into the report.
Will only this instance get the key?	The report binds a transport key the receiver generated at startup; the key is sealed to it.
Is this the current policy?	The policy epoch is bound into the report, and an optional owner epoch store rejects older policies across owner restarts.
Is the platform acceptable?	Guest policy, VMPL 0, component security-version floors, and required or forbidden platform flags are checked.

2. Related work

Attestation-conditioned key release is established. AWS KMS key policies can require that a Nitro Enclave attestation document carry a given image digest or PCR value before `Decrypt` or `GenerateDataKey` succeeds [1]. The Confidential Containers project's Trustee releases secrets to guests under a resource policy evaluated over attestation results [2]. This report does not claim the general idea of requiring a measurement before releasing a secret.

Confidential workload systems also attest the component that makes the release decision. In Contrast, a coordinator running in a confidential VM verifies each pod's attestation against a manifest and issues service-mesh certificates [3]. The question studied here is narrower: an owner approving the broker's effective configuration, fresh receiving key and policy epoch, and the lifecycle that follows. The RATS architecture [4] gives the roles used below. The owner is the relying party and verifier for the broker, and the broker is in turn the verifier for workloads.

Predicting an SEV-SNP launch measurement from build inputs is done by tools such as `sev-snp-measure` [5]. The deployment approval in section 4 records such a prediction; it does not compute one. Rollback of the owner's epoch store is the same state-continuity problem that Memoir [6] and ROTE [7] address.

3. Participants and trust

The owner controls the model key, the owner signing key, the trust roots, the provisioning policy and the deployment approval. The broker receiver generates a fresh X25519 transport key when it starts. A hardware report provider produces an attestation report over the owner's challenge and a binding computed from the receiver's key and the policy.

An adversary may substitute report bytes, the receiving public key, the model identity, the configuration, a build artifact or the policy version. It may replay a report or an envelope, request provisioning twice, or present an unsuitable platform with a validly signed report. The owner has to tell a genuinely signed report from an acceptable one.

The owner's policy process and its key storage are trusted. A deployed broker additionally needs measured code, immutable effective inputs, a trustworthy clock and memory isolation. The Python library cannot enforce those properties by itself.

4. Protocol and lifecycle

The owner's policy names the expected launch measurement, the configuration digest, the model weights hash and the epoch. It also sets the guest policy (debug-enabled policies are refused), a minimum security version for each TCB component, and required or forbidden platform flags.

The configuration digest covers the receiver's actual constructor inputs: the CPU root and VCEK certificates, any intermediates, the exact allowlist of manifest identities, the owner public key, an optional GPU root, and the fixed security settings of the profile, such as required channel binding and challenge lifetimes. The receiver computes it from what it was given and refuses construction if it differs from the owner policy. It is not a caller-supplied description.

```
owner-approved artifacts and launch inputs
|
fresh broker key + effective configuration + policy epoch
|
owner nonce -> report binding -> signed report appraisal
|
owner-authenticated sealed provisioning envelope
|
receiver: awaiting -> pending -> provisioned -> retired
```

The report data is `SHA256(nonce || binding)`, where the binding holds a domain separator, the fresh transport public key and a hash of the full policy context. The owner consumes the challenge first, then checks the certificate chain, the report signature and the binding, and only then checks the measurement, report version, guest policy, VMPL, TCB floors and platform flags. A correctly signed report with the wrong measurement is rejected.

After appraisal, the owner seals the model key to the receiver's transport key with the policy context, nonce and receiving public key as associated data, and signs the result. The receiver checks the pending nonce and its own installation deadline of at most 60 seconds. Any installation attempt consumes the pending request. A successful install builds the restricted key broker and drops the reference to the provisioning private key. Provisioning the same active instance again is refused.

Retirement marks the receiver permanently unavailable through its API and drops its references to the broker and any pending state. A new receiver needs fresh provisioning. Policy updates must advance the epoch and invalidate outstanding owner challenges. An optional owner epoch store, a SQLite table keyed by policy lineage, holds the floor across owner processes; ordinary operations require an exact epoch and digest match, and admission of a higher epoch happens inside one write transaction.

The deployment approval records the hash and size of the firmware, kernel, initrd, command line, application, configuration and measurement tool, together with the launch parameters, an independently predicted launch measurement, the configuration digest and the policy digest. The owner pins the approval's identity before deployment, and a provisioner built with an approval refuses any policy the approval does not cover. The approval helper checks consistency among these records. It does not compute the SNP launch measurement, and it never learns an expected measurement from a report supplied by the deployment.

5. Security reasoning

Binding the receiver's key means a valid report cannot provision a different receiver. Binding the policy context means the same approval cannot silently change configuration, model identity or epoch. Hashing the actual constructor inputs makes configuration substitution visible before the receiver can be provisioned at all.

Consuming a challenge before appraisal limits each challenge to one attempt, including failed ones. That costs retry convenience after a transient failure; the owner issues a fresh challenge instead of reopening consumed state.

All of this depends on the link between the signed report and the code that holds the private key. If the measured program later loads attacker-controlled code or exports its state, the protocol cannot help.

6. Evaluation

The evaluated revision is Weight Custody Manifest [db8f106](#), merged in [WCM PR 148](#) on September 20, 2026. The four modules studied here (`provisioning.py`, `broker_receiver.py`, `provisioning_state.py`, `deployment_identity.py`) ship byte for byte in `weight-custody-manifest` 0.28.4 and 0.29.0 on PyPI. The experiments were rerun on October 1, 2026 from a fresh virtual environment with Python 3.12.10, `pytest` 9.1.1, `cryptography` 50.0.1 and `pydantic` 2.13.5, against the evaluated revision and against current main, [5ef8c12](#). Both runs gave the numbers below, which also match the values first recorded on September 23 and 27, 2026.

6.1 Existing tests and control removals

Three suites cover owner provisioning, the broker receiver and deployment approval. All **84 tests** passed. They use generated certificate chains and signed SNP-format reports, and cover valid provisioning and opening, report and transport-key substitution, replay, expiry, concurrent reuse of one challenge, platform requirements, policy updates, receiver restart, retirement, failed installation, changed effective inputs and modified deployment artifacts. The tests and the implementation share some supporting library code, so this is not an independent attestation implementation.

Change under test	Failing tests	Tests that caught it
Receiver skips the effective-configuration comparison	5 of 84	one per changed input: CPU root, VCEK, owner key, manifest allowlist, GPU root
Owner skips the measurement and report-version check	3 of 84	image substitution; report snapshot before authentication; identity-constrained provisioning with a wrong measurement

These show the suites depend on the two controls. They do not show that every configuration or measurement error is covered.

6.2 Receiver retirement across a process boundary

A `BrokerReceiver` runs in its own process and is driven over a line protocol by an owner process. The owner provisions it with a random 32-byte model key, and a workload release succeeds: the sealed key opens to the same bytes. After `retire()`, all four API paths refused with `RuntimeError`: report data for a fresh owner challenge, installing the previous envelope, issuing a challenge and releasing a key. A replacement receiver process refused the old envelope with `ValueError`: its nonce belongs to the earlier request, and it is sealed to the old receiver's key. Only fresh provisioning let the replacement serve.

The experiment also reads the retired process's committed memory from outside with Windows `ReadProcessMemory` and counts copies of the model key. One copy was present before retirement, and one copy was still present after retirement and garbage collection. A random 32-byte value that was never loaded was found 0 times, so the scan is not producing false matches. Retirement is an API state. Dropping Python references does not erase key material, and this measurement shows it.

6.3 Owner epoch recovery across processes

Separate owner processes share one `OwnerEpochStore` file. Each step starts a new process, except that one epoch 1 owner stays alive across a policy update.

Step	Result	Floor after
Epoch 1 owner starts and provisions	provisioned	1
Epoch 2 owner admits a new policy and provisions	provisioned	2
Epoch 1 owner that was already running provisions after the update	refused	2
Epoch 1 owner restarts	refused at construction	2
Epoch 2 owner with a different configuration digest	refused at construction	2
Epoch 2 owner restarts with the admitted policy	provisioned	2
Epoch 3 admission, process killed inside the floor transaction	process exit 73	2
Epoch 2 owner after the killed admission	provisioned	2
Epoch 3 owner admits normally	provisioned	3

All 9 steps matched. The killed admission shows that an interrupted update neither advances the floor nor blocks the current policy. Restoring a copy of the store taken at epoch 1, or deleting the store, let an epoch 1 owner provision again. That is the rollback limit the store's own documentation states.

6.4 Controls

Each part has a control that must fail. Replacing `self._retired = True` with `False` in an isolated copy let the retired receiver produce report data for a fresh challenge again, so the retirement criterion failed (9 of 10 cases passed). Replacing the floor comparison with `if False:` let the stale live owner, the restarted epoch 1 owner and the same-epoch substitution all provision, so 3 of the 9 steps failed (7 of 10 cases passed).

7. Limits

The open deployment question is whether approved build inputs determine the protected receiver that actually runs. The hardware experiment that answers it would predict a launch measurement independently, obtain a fresh report from that launch, and change one build artifact, one configuration field and one receiver key at a time. Each changed case must fail before the owner emits an envelope, while the unchanged control succeeds. That needs SEV-SNP hardware and a launch-measurement tool, and it has not been run.

The epoch store must live on owner-controlled storage with rollback protection; section 6.3 shows what restoring or deleting it does. Secure erasure of retired key material is not established, and section 6.2 shows it does not happen through the Python API. All runs used one Windows host; no throughput or provisioning-latency figure is claimed.

Changes on main since the evaluated revision do not touch the four modules. Later changes do affect code the receiver uses after installation: [WCM PR 163](#) made the shared challenge store thread-safe and bounded its memory, and PRs [156](#), [162](#), [154](#) and [170](#) tightened workload key release (GPU confidential-compute mode, manifest-required evidence verification, nonce-bound OCSP, and comparison of the signed launch measurement). The owner path already serialized challenge consumption under its own lock at the evaluated revision. No result in this report changes at main.

8. Conclusion

The tested software binds owner approval to one provisioning context, exposes configuration, measurement, key and epoch substitutions before the model key leaves the owner, stops serving after retirement, and holds an owner epoch floor across process restarts and an interrupted admission. A team can check this for itself: the 84 tests, the two control removals and the lifecycle experiment with its two controls rerun with one command against any Weight Custody Manifest clone. The hardware half of the claim, that an approved image is what runs, still needs the launch experiment in section 7.

The same provisioning lifecycle appears as one component of the broader report *Confidentiality Across Agent Handoffs* (OPAQUE Systems technical report, version 2, September 30, 2026, DOI [10.5281/zenodo.23074670](https://doi.org/10.5281/zenodo.23074670)). This report is the focused study with its own pinned evaluation; it does not change the results of that one.

Appendix A. Reproduction

Item	Value
Evaluated revision	Weight Custody Manifest db8f10638bb9d6dac7b1e115b7760dffc9e2635d
Current main rerun	Weight Custody Manifest 5ef8c122479f5df79ad8a7c04b70f4a43bbdb313
Released package	weight-custody-manifest 0.28.4 and 0.29.0 contain the four evaluated modules unchanged
Environment	Windows 11, Python 3.12.10, pytest 9.1.1, cryptography 50.0.1, pydantic 2.13.5
Command	<code>python -B experiments/rerun.py --repo <weight-custody-manifest clone> --output <new directory> [--revision <commit>]</code>
Records	experiments/results/rerun-2026-10-01/evaluated-db8f106/ and main-5ef8c12/, each with summary.json, JUnit XML, logs, experiment JSON and SHA256SUMS.json

The runner exports the chosen revision with `git archive`, runs the three suites, the two control removals, and `lifecycle.py` with its two controls, then compares every count with the values recorded when the report was written. `lifecycle.py` reuses the test suite's synthetic certificate chain and report helpers. The memory scan runs only on Windows; elsewhere it records that the platform is unsupported. Local paths are replaced by placeholders before anything is written.

Implementation at the evaluated revision: [owner provisioning](#), [broker receiver](#), [owner epoch store](#), [deployment approval](#).

References

- [1] Amazon Web Services. [Condition keys for Nitro Enclaves](#). AWS Key Management Service Developer Guide.
- [2] Confidential Containers. [Trustee policies](#). Confidential Containers documentation.
- [3] Edgeless Systems. [Contrast attestation overview](#). Contrast documentation.
- [4] H. Birkholz, D. Thaler, M. Richardson, N. Smith and W. Pan. [Remote ATtestation procedureS \(RATS\) Architecture](#). RFC 9334, January 2023.
- [5] VirTEE. [sev-snp-measure](#). Command-line tool and library for calculating SEV, SEV-ES and SEV-SNP launch measurements.
- [6] B. Parno, J. R. Lorch, J. R. Douceur, J. Mickens and J. M. McCune. [Memoir: Practical State Continuity for Protected Modules](#). IEEE Symposium on Security and Privacy, 2011.
- [7] S. Matetic, M. Ahmed, K. Kostiainen, A. Dhar, D. Sommer, A. Gervais, A. Juels and S. Čapkun. [ROTE: Rollback Protection for Trusted Execution](#). USENIX Security Symposium, 2017.